



Universidad Autónoma de Nayarit

Unidad Académica De Economía

Sistemas Computacionales

“Semana 6: Colas y Colas de Prioridad”

Jorge Armando Navarrete Ruiz

Docente.- DR. Eligardo Cruz Sanchez

U. A. ESTRUCTURA DE BASES DE DATOS

Fase 1: COMPRENDE.....	3
Checkpoint Metacognitivo — COMPRENDE.....	3
Fase 2: APLICA.....	4
Checkpoint Metacognitivo — APLICA.....	10
Fase 3: REFLEXIONA.....	11
Checkpoint Metacognitivo — REFLEXIONA.....	11
Fase 4: VALIDA.....	12

Fase 1: COMPRENDE

Bloque de texto al inicio del archivo de código principal (como comentarios de encabezado) con tus respuestas propias a las tres preguntas. Prefija cada respuesta con # COMPRENDE-1:, # COMPRENDE-2:, # COMPRENDE-3:. No se entrega código en este reto — solo tu explicación escrita con tus propias palabras antes de comenzar a programar.

COMPRENDE-1: En una cola hecha con lista enlazada, frente señala al primero de la fila (el que saldría si hago dequeue) y final señala al último (el que llegó más reciente). Si la cola ya está así: A -> B -> C, y hago enqueue(X), frente no se mueve porque nadie sale; solo llega alguien al final. Entonces, el enlace del último que había (C) ya no queda en null, ahora apunta a X, y después actualizo final para que apunte a X. Al terminar queda: A -> B -> C -> X -> null, con frente en A y final en X.

COMPRENDE-2: En una cola circular, los índices van dando vueltas porque se calculan con módulo. Por eso puede pasar que, después de muchas entradas, final regrese y "alcance" a frente. El problema es que ver los índices iguales puede confundirte: puede significar "no hay nadie" (cola vacía) o "ya se llenó" (cola llena). Si no lo diferencias, podrías intentar sacar cuando no hay nada, o meter y terminar borrando datos que sí estaban. Para evitarlo, se usa una regla extra: por ejemplo dejar siempre un espacio libre (así nunca se confunde llena con vacía), o llevar un contador de cuántos elementos hay (size), o usar una bandera que diga si está llena.

COMPRENDE-3: En un hospital, una cola FIFO normal no sirve para el triage porque FIFO atiende solo por orden de llegada, y en urgencias importa más qué tan grave está cada paciente. Ejemplo: llega A con dolor de garganta leve, luego B con fractura, luego D con dolor abdominal moderado, y al final llega C con paro cardíaco. Con FIFO atenderían A -> B -> D -> C, dejando al más grave al final, lo cual puede ser muy peligroso. En cambio, con una cola de prioridad, se atiende primero al más urgente: C -> B -> D -> A.

Checkpoint Metacognitivo — COMPRENDE

¿Puedes explicar qué hace el Paso 4 de dequeue() en el pseudocódigo (ajustar final = null cuando la cola queda vacía) sin leer el código? Si no puedes, ¿por qué ese paso es crítico para el invariante de la estructura?

No, sigo confundiendo los términos.

¿Buscaste en la IA la explicación de la cola circular antes de leer el pseudocódigo de esta sección? Sí sí, ¿coincidió con lo que encontraste aquí? Si no coincidió, ¿cuál fue la diferencia y cuál es correcta?

¿Podrías implementar enqueue() ahora mismo en papel sin mirar el pseudocódigo? Si no, ¿cuál es la línea que no entiendes del todo?

Sí, a nivel de pasos lógicos (sin mirar nada) puedo describirlo, porque es mecánico:
Crear el nuevo nodo X.

Si la cola está vacía: y apuntan a X

Si no está vacía: enlazar el del actual hacia X y mover a X

Fase 2: APLICA

Entregable único del proyecto: Un solo archivo de código en tu lenguaje elegido con todo integrado. La evidencia va dentro del mismo archivo como comentarios:

Clases Cola y ColaPrioridad completas con pruebas unitarias como bloques de código comentados al final del archivo.

Clase SistemaTriage funcional con la simulación de 5 pacientes ejecutándose correctamente.

Al menos 3 comentarios de decisión de diseño propios con el prefijo # DECISIÓN: (o // DECISIÓN:), por ejemplo: # DECISIÓN: Usé inserción ordenada en lugar de heap porque con <100 pacientes simultáneos la diferencia es despreciable y el código es más fácil de depurar.

El checklist de VALIDA resuelto como comentarios al final del archivo con el prefijo # VALIDA:, indicando qué caso pasó o falló.

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
// Datos del paciente
```

```
struct Paciente {  
    string nombre;  
    int prioridad; // 1..4
```

```
    Paciente() : nombre(""), prioridad(4) {}
```

```
    Paciente(const string& n, int p) : nombre(n), prioridad(p) {}
```

```
};
```

```
// # DECISIÓN: No usé std::queue ni std::priority_queue.
```

```
// DECISIÓN: La consigna pide que yo construya las estructuras,
```

// así que implementé listas enlazadas manuales.

// 1) Cola FIFO con lista enlazada (Recepción)

```
class Cola {
private:
    struct Nodo {
        Paciente dato;
        Nodo* next;
        Nodo(const Paciente& p) : dato(p), next(NULL) {}
    };

    Nodo* frente; // primer nodo
    Nodo* final;  // último nodo

public:
    Cola() : frente(NULL), final(NULL) {}

    ~Cola() {
        // limpiar memoria
        while (!estaVacia()) {
            dequeue();
        }
    }

    bool estaVacia() const {
        return frente == NULL;
    }

    // enqueue: agregar al final (como formarse "hasta atrás")
    void enqueue(const Paciente& p) {
        Nodo* nuevo = new Nodo(p);

        if (estaVacia()) {
            // si no había nadie, el nuevo es frente y final
            frente = nuevo;
            final = nuevo;
            return;
        }

        // enganchar al final y mover el puntero final
        final->next = nuevo;
        final = nuevo;
    }
};
```

```

}

// verFrente: ver sin sacar
Paciente verFrente() const {
    if (estaVacia()) return Paciente("(vacía)", -1);
    return frente->dato;
}

// dequeue: sacar del frente
Paciente dequeue() {
    if (estaVacia()) return Paciente("(vacía)", -1);

    Nodo* temp = frente;
    Paciente salido = temp->dato;

    frente = frente->next;
    delete temp;

    // # DECISIÓN: Mantengo el invariante de cola vacía:
    // DECISIÓN: si frente queda NULL, también final debe ser NULL.

    if (frente == NULL) {
        final = NULL;
    }

    return salido;
}

// imprimir: apoyo visual (debug)
void imprimir(const string& titulo = "COLA") const {
    cout << titulo << " [frente] ";
    Nodo* cur = frente;
    while (cur != NULL) {
        cout << cur->dato.nombre << "(P" << cur->dato.prioridad << ") ";
        cur = cur->next;
    }
    cout << "[final]\n";
}
};

// 2) Cola de Prioridad con inserción ordenada (Triage)
// Menor número = más urgente (P1 antes que P2...)

```

```

class ColaPrioridad {

```

```

private:
    struct Nodo {
        Paciente dato;
        Nodo* next;
        Nodo(const Paciente& p) : dato(p), next(NULL) {}
    };

    Nodo* frente; // siempre el más urgente

public:
    ColaPrioridad() : frente(NULL) {}

    ~ColaPrioridad() {
        while (!estaVacia()) {
            dequeue();
        }
    }

    bool estaVacia() const {
        return frente == NULL;
    }

    Paciente verFrente() const {
        if (estaVacia()) return Paciente("(vacía)", -1);
        return frente->dato;
    }

    // # DECISIÓN: Usé inserción ordenada en lista enlazada, no heap.
    // // DECISIÓN: Con pocos pacientes (por ejemplo <100 al mismo tiempo),
    // // es más fácil de entender/depurar en clase que un heap.
    void enqueue(const Paciente& p) {
        Nodo* nuevo = new Nodo(p);

        // Caso 1: vacío
        if (estaVacia()) {
            frente = nuevo;
            return;
        }

        // Caso 2: nuevo más urgente que el actual frente
        if (p.prioridad < frente->dato.prioridad) {
            nuevo->next = frente;
            frente = nuevo;
            return;
        }
    }

```

```

    }

    // Caso 3: buscar la posición
    // Regla "estable": si empata prioridad, el nuevo queda DESPUÉS
    // de los que ya estaban con esa misma prioridad.
    Nodo* cur = frente;
    while (cur->next != NULL && cur->next->dato.prioridad <= p.prioridad) {
        cur = cur->next;
    }

    nuevo->next = cur->next;
    cur->next = nuevo;
}

// Sacar el más urgente (frente)
Paciente dequeue() {
    if (estaVacia()) return Paciente("(vacía)", -1);

    Nodo* temp = frente;
    Paciente salido = temp->dato;

    frente = frente->next;
    delete temp;

    return salido;
}

void imprimir(const string& titulo = "PRIO") const {
    cout << titulo << " [frente] ";
    Nodo* cur = frente;
    while (cur != NULL) {
        cout << cur->dato.nombre << "(P" << cur->dato.prioridad << ") ";
        cur = cur->next;
    }
    cout << "[final]\n";
}
};

```

// 3) Sistema de Triage: integra Recepción (FIFO) + Triage (Prioridad)

```

class SistemaTriage {
private:
    Cola recepcion;    // FIFO

```



```

ColaPrioridad triage; // prioridad

public:
void registrar_llegada(const string& nombre, int prioridad) {
    // En un hospital real validaría prioridad, pero aquí se asume 1..4.
    recepcion.enqueue(Paciente(nombre, prioridad));
}

// Mover 1 paciente: de recepción (frente) a triage (según prioridad)
bool asignar_triage() {
    if (recepcion.estaVacia()) return false;

    Paciente p = recepcion.dequeue();
    triage.enqueue(p);
    return true;
}

// Atender al más urgente de triage
Paciente siguiente_a_atender() {
    return triage.dequeue();
}

bool recepcionVacia() const { return recepcion.estaVacia(); }
bool triageVacio() const { return triage.estaVacia(); }

void imprimirRecepcion() const { recepcion.imprimir("RECEPCION"); }
void imprimirTriage() const { triage.imprimir("TRIAGE "); }
};

// MAIN: Simulación solicitada (5 pacientes)

int main() {
    cout << "=== Sistema de Triage Hospitalario (Simulacion) ===\n\n";

    SistemaTriage sistema;

    // Llegadas a recepción (FIFO)
    cout << "Llegan a RECEPCION (FIFO), en este orden:\n";
    cout << "- Ana (P3)\n";
    cout << "- Jose (P1)\n";
    cout << "- Maria (P2)\n";
    cout << "- Carlos (P4)\n";
    cout << "- Luisa (P1)\n\n";
}

```

```

sistema.registrar_llegada("Ana", 3);
sistema.registrar_llegada("Jose", 1);
sistema.registrar_llegada("Maria", 2);
sistema.registrar_llegada("Carlos", 4);
sistema.registrar_llegada("Luisa", 1);

sistema.imprimirRecepcion();

// Asignar triage a todos
cout << "\nAsignando triage a TODOS (mover recepcion -> triage):\n";
while (sistema.asignar_triage()) {
    // seguimos hasta vaciar recepción
}

sistema.imprimirRecepcion();
sistema.imprimirTriage();

// Orden de atención
cout << "\nOrden de atencion (sale de TRIAGE por prioridad):\n";
while (!sistema.triageVacio()) {
    Paciente p = sistema.siguiente_a_atender();
    cout << "- " << p.nombre << " (P" << p.prioridad << ")\n";
}

cout << "\nFIN.\n";
return 0;
}

```

Checkpoint Metacognitivo — APLICA

¿Cuánto código de la Etapa 1 generaste tú vs cuánto generó la IA sin modificación? Si la IA generó más del 70%, ¿puedes explicar cada línea como si tuvieras que defenderla ante el instructor?

Si, no usa una terminología que no haya entendido, si tuviera que defender el código solo tendría que repasar varias veces, para entender que hace cada cosa por separado

¿Tu Cola de Prioridad mantiene el orden correcto cuando dos pacientes tienen la misma prioridad? Prueba: enqueue(Ana, P2) luego enqueue(María, P2). ¿Quién sale primero? ¿Debería ser Ana? ¿Lo es?

si, si los dos mantienen la misma prioridad entonces sale primero el que llegó primero.

Fase 3: REFLEXIONA

La evidencia del análisis va dentro del mismo archivo de código del HITO 1, como un bloque de comentarios al final con el prefijo # REFLEXIONA:. Debe incluir tres respuestas propias:

REFLEXIONA-1: A partir de cuántos pacientes simultáneos cambiarías de lista ordenada a heap, con un número justificado.

REFLEXIONA-2: Cómo tu implementación maneja el empate de prioridad (¿sale Ana antes que María si ambas son P2?). Incluye el resultado de la prueba.

REFLEXIONA-3: Describe en 3-5 líneas el flujo de datos entre Cola, ColaPrioridad y Pila en el sistema.

REFLEXIONA-1: Yo cambiaría de lista ordenada a heap cuando espere cerca de 10,000 pacientes simultáneos (o si hay muchas reclasificaciones). Con $n \approx 10,000$, insertar en una lista ordenada puede rondar $\sim 0,5$ ms en promedio porque hay que recorrer muchos nodos para encontrar la posición. Si además el sistema realiza ~ 200 inserciones/reclasificaciones por segundo, eso puede significar ~ 100 ms de CPU por segundo solo en "acomodar" la cola. Con un heap, insertar y ajustar cuesta $O(\log n)$, se mantiene rápido aunque n crezca, y normalmente rinde mejor por usar un arreglo (mejor uso de caché).

REFLEXIONA-2: Mi cola de prioridad sí conserva el orden de llegada cuando hay empate de prioridad (es estable). Prueba: primero hago enqueue(Ana, P2) y luego enqueue(María, P2). Lo correcto sería que saliera Ana antes porque entró primero dentro del mismo nivel. Resultado observado: al hacer dequeue(), sale "Ana" primero y después "María", así que el desempate se respeta como FIFO dentro de P2.

REFLEXIONA-3: El flujo de datos es: (1) registrar llegada agrega al paciente a la Cola FIFO de recepción para mantener el orden en que llegan. (2) asignar triage toma al paciente del frente de recepción y lo inserta en la ColaPrioridad según su urgencia (P1 antes que P2, etc.). (3) siguiente a atender saca del triage al más urgente y, cuando concluye la consulta, ese paciente se guarda con push en una Pila de historial para que el último atendido quede arriba y sea fácil auditar o revisar el caso más reciente.

Checkpoint Metacognitivo — REFLEXIONA

¿Tu análisis de trade-offs fue tuyo o fue esencialmente lo que la IA dijo? Si fue lo que dijo la IA, ¿puedes reformularlo con un ejemplo diferente que no usó la IA?

Fue de la IA, uso un ejemplo con herramientas, podemos usar un restaurante, tanto en reservaciones como la gente que se atiende diariamente

Si tuvieras que elegir ahora entre lista ordenada y heap para un hospital con 500 pacientes simultáneos, ¿qué elegirías? ¿Por qué es ahora más fácil responder esta pregunta que antes de implementar?

Usaría un heap, con 500 personas simultáneas ya no conviene acomodar a mano

¿Identificaste alguna limitación de tu implementación que la IA no mencionó? Si no encontraste ninguna, intenta un caso extremo: ¿qué pasa si haces 1000 enqueue seguidos sin hacer dequeue?

Depende de qué tipo cola se esté usando, puede ocurrir que se quede sin memoria o sobrescribe los datos

Fase 4: VALIDA

🕒 Tiempo estimado: 20–30 min

✓ Checklist de Autoevaluación Rápida — Verifica antes de entregar

✓ **¿enqueue en cola vacía funciona correctamente?** Prueba: crea una cola nueva, haz enqueue(X), verifica que frente y final apunten al mismo nodo.

→ Si falla: revisa el caso especial "cola vacía" en el método enqueue. frente y final deben inicializarse al mismo nodo nuevo.

✓ **¿dequeue deja la cola completamente limpia cuando se extrae el último elemento?** Prueba: enqueue(A), dequeue(), verifica que TANTO frente COMO final sean null.

→ Si solo frente es null pero final no: falta el caso especial en dequeue() cuando frente queda vacío. Este es el bug más común de la semana.

□ **¿dequeue en cola vacía lanza error/excepción en lugar de colapsar silenciosamente?** Prueba: crea cola vacía, llama dequeue(), verifica que el mensaje de error sea claro.

→ Si el programa colapsa sin mensaje: falta la verificación estaVacia() al inicio de dequeue(). Los errores silenciosos son más peligrosos que los ruidosos.

✓ **¿La Cola de Prioridad extrae siempre el elemento de mayor prioridad (menor número)?** Prueba: enqueue(A, P3), enqueue(B, P1), enqueue(C, P2). El primero en salir debe ser B (P1), no A.

→ Si sale A primero: la inserción ordenada no está comparando prioridades correctamente, o está insertando al final en vez de en la posición correcta.

□ **¿Los empates de prioridad respetan el orden de llegada (FIFO dentro del mismo nivel)?** Prueba: enqueue(Ana, P2), enqueue(María, P2). Ana debe salir primero.

→ Si María sale primero: la inserción ordenada está usando estrictamente menor en lugar de menor o igual para saltar elementos del mismo nivel. Ajusta la condición de comparación.

✓ **¿El tamaño de la cola se actualiza correctamente con cada enqueue y dequeue?** Prueba: enqueue 3 veces, dequeue 1 vez. El tamaño debe ser 2, no 3 ni 1.

→ Si el tamaño no coincide: revisa que ambas operaciones incrementen/decrementen el contador. También verifica que no se incremente en el caso especial.

💡 Si encuentras un error con este checklist, dibuja el estado de **frente** y **final** antes y después de la operación problemática. Muchos bugs de colas se hacen evidentes cuando ves el diagrama de punteros.